Proyecto Integrador

AsistenteIA



UNIVERSIDAD DE SAN BUENAVENTURA

Higuera Perez Juan Felipe

Muñoz Solorzano Cristian Fabian

Facultad de Ingeniería, Optativa II

Ing. Jairo Salcedo

4 de Noviembre del 2026

I. INTRODUCCIÓN

En los proyectos de desarrollo de software, la documentación técnica permite comunicar el propósito del sistema, sus requisitos, la organización de sus componentes y las decisiones que orientan su implementación. También facilita el mantenimiento, las pruebas y la identificación temprana de riesgos.

II. ALCANCE Y FUNCIONALIDADES PRINCIPALES

AsistenteIA permite formular consultas a un servicio de inteligencia artificial, utilizar plantillas, recibir respuestas y guardar conversaciones en un historial local. Sus secciones principales son Inicio, Consultar, Historial y Configuración.

El sistema no implementa un servidor backend propio. El cliente móvil se comunica directamente con Pollinations mediante HTTPS. El historial se conserva en SQLite en el dispositivo y la clave API se almacena mediante SecureStorage.

III. PARTES INTERESADAS Y PREOCUPACIONES

Las partes interesadas principales son los usuarios de la aplicación, el equipo de desarrollo y el docente evaluador. Los usuarios necesitan una interfaz comprensible, protección de sus credenciales y respuestas útiles. El equipo de desarrollo requiere componentes separados y mantenibles. El docente evaluador necesita una descripción verificable del alcance.

Las preocupaciones relevantes incluyen la confidencialidad de las claves API, el envío de consultas a un proveedor externo, la exactitud de las respuestas, la disponibilidad de Internet y la conservación segura del historial local.

IV. OBJETIVOS

Objetivo general: documentar los requisitos, la arquitectura, las decisiones técnicas y el comportamiento de integración REST de AsistenteIA, junto con los riesgos y las instrucciones básicas de uso y mantenimiento.

Objetivos específicos: identificar las funciones y restricciones del sistema; describir sus componentes y flujo de datos; documentar la solicitud, respuesta y códigos HTTP; analizar

riesgos de privacidad, seguridad y uso responsable; y establecer instrucciones de operación e instalación.

ÍNDICE

Control del documento..... 4
V. SISTEMA DE INTERÉS..... 5
5.1 Funciones identificadas..... 5
5.2 Actores..... 5
VI. REQUISITOS DEL SISTEMA..... 6
6.1 Requisitos funcionales..... 6
6.2 Requisitos no funcionales..... 7
6.3 Restricciones y supuestos..... 7
VII. ARQUITECTURA DE SOFTWARE..... 8
7.1 Vista lógica..... 8
7.2 Componentes y responsabilidades..... 8
7.3 Flujo de una consulta..... 9
7.4 Persistencia y datos..... 9
VIII. DECISIONES TÉCNICAS..... 10
IX. CONSUMO DEL SERVICIO REST..... 11
5.1 Solicitud..... 11
5.2 Respuesta exitosa esperada..... 11
5.3 Códigos HTTP y tratamiento..... 12
5.4 Encabezados y manejo de errores..... 13
5.5 Consideraciones de contrato..... 13
VIII. MATRIZ DE RIESGOS DE PRIVACIDAD, SEGURIDAD Y USO RESPONSABLE..... 13
8.1 Reglas de uso responsable para el usuario..... 16
IX. MANUAL DE USUARIO..... 16
9.1 Requisitos previos..... 16
9.2 Primer uso: configurar la clave..... 17
9.3 Realizar una consulta..... 17
9.4 Consultar y administrar el historial..... 18
9.5 Eliminar la clave o cambiar el modelo..... 18
9.6 Mensajes frecuentes..... 18
X. MANUAL TÉCNICO BREVE..... 19
10.1 Stack tecnológico..... 19
10.2 Estructura relevante del proyecto..... 19
10.3 Entorno de desarrollo..... 20
10.4 Instalación y compilación..... 20

10.5 Configuración de servicios..... 20
10.6 Almacenamiento local..... 21
10.7 Puntos de mantenimiento.....21
XI. PRUEBAS Y CRITERIOS DE ACEPTACIÓN..... 21
XII. 10 Pruebas de la app.....22
XIII. REFERENCIAS.....27

Resumen— Este documento presenta la documentación técnica de AsistenteIA, una aplicación académica desarrollada con .NET MAUI que permite enviar consultas a un servicio de inteligencia artificial mediante una API REST, recibir respuestas y conservar conversaciones en un historial local. Se describen los requisitos funcionales y no funcionales, la arquitectura basada en MVVM, las decisiones técnicas, el contrato de consumo REST, los riesgos de privacidad, seguridad y uso responsable, y los manuales de usuario y técnico. La documentación se elaboró a partir del repositorio del proyecto, diferenciando las funcionalidades implementadas de las recomendaciones de mejora.

Palabras clave— .NET MAUI, MVVM, API REST, Pollinations, SQLite, inteligencia artificial.

Control del documento

Campo	Detalle
Nombre del sistema	AsistenteIA
Tipo de solución	Aplicación móvil multiplataforma desarrollada con .NET MAUI.
Repositorio	Cristianm23/AsistenteIA, rama predeterminada master.
Versión de documentación	1.0
Fecha de análisis	5 de noviembre de 2026
Alcance	Código y documentación disponibles en el repositorio público consultado.

Criterio de precisión Este documento describe el comportamiento observado en el repositorio. Los endpoints, estructuras JSON y estados HTTP se documentan con base en el cliente implementado y el contrato compatible con Chat Completions. Los códigos de error del proveedor pueden variar. Las recomendaciones futuras se identifican como tales.

V. SISTEMA DE INTERÉS

AsistenteIA es una aplicación académica que permite formular preguntas a un servicio de inteligencia artificial, recibir respuestas, utilizar plantillas de consulta y conservar conversaciones en un historial local. Está orientada al apoyo del aprendizaje: explicar conceptos, resumir temas, comparar ideas, generar ejemplos y preparar preguntas de práctica.

La aplicación no implementa un backend propio. El cliente móvil se comunica directamente con la API de Pollinations mediante HTTPS y almacena el historial en una base de datos SQLite dentro del dispositivo. La clave de API se conserva mediante SecureStorage y la selección del modelo mediante Preferences.

5.1 Funciones identificadas

- Inicio: presenta el propósito, recomendaciones de uso y accesos a consulta e historial.
- Consulta IA: permite escribir un prompt, seleccionar una plantilla, enviar la solicitud, cancelar una operación y guardar la respuesta.
- Plantillas: incluye opciones para explicar, resumir, crear ejemplos, practicar y comparar conceptos.
- Historial: lista conversaciones guardadas, permite búsqueda, apertura de detalle y eliminación.
- Detalle: muestra la consulta, respuesta, fecha y categoría; permite marcar la respuesta como verificada y guardar ese estado.
- Configuración: registra o elimina la clave API, define el identificador del modelo y prueba la conectividad con el proveedor.

5.2 Actores

Actor	Responsabilidad
Usuario	Configura el acceso al proveedor, envía consultas, revisa respuestas y administra su historial.
API de Pollinations	Recibe solicitudes autenticadas y devuelve contenido generado por el modelo seleccionado.
Sistema operativo	Proporciona almacenamiento seguro, preferencias, conectividad y el entorno de ejecución móvil.

VI. REQUISITOS DEL SISTEMA

6.1 Requisitos funcionales

ID	Requisito	Descripción
RF-01	Navegación	La aplicación debe ofrecer las secciones Inicio, Consultar, Historial y Configuración.
RF-02	Validación de consulta	Debe aceptar consultas entre 5 y 120 caracteres después de eliminar espacios iniciales y finales.
RF-03	Autenticación REST	Debe enviar la clave configurada en el encabezado Authorization como Bearer token.
RF-04	Generación de respuesta	Debe enviar el modelo y los mensajes system/user al endpoint de chat completions y mostrar el primer contenido devuelto.
RF-05	Plantillas	Debe permitir seleccionar una plantilla y cargar su texto en el campo editable de consulta.
RF-06	Cancelación	Debe permitir cancelar la solicitud en curso mediante CancellationToken.
RF-07	Persistencia	Debe guardar localmente prompt, respuesta, fecha UTC, categoría y estado de verificación.
RF-08	Consulta del historial	Debe listar los registros, ordenados por fecha descendente, y permitir buscar en prompt o respuesta.

RF-09	Administración del historial	Debe permitir consultar el detalle, cambiar el indicador de verificación y eliminar registros.
RF-10	Gestión de credenciales	Debe guardar, consultar, indicar presencia y eliminar la clave mediante SecureStorage sin mostrar su valor.
RF-11	Configuración de modelo	Debe guardar el identificador del modelo en Preferences y recuperarlo al iniciar la consulta.
RF-12	Prueba de conexión	Debe ofrecer una prueba enviando una consulta breve al servicio de IA.
RF-13	Estado de operación	Debe informar al usuario sobre carga, falta de conexión, cancelación, errores y resultado de la operación.

6.2 Requisitos no funcionales

ID	Atributo	Descripción
RNF-01	Plataforma	Aplicación .NET MAUI con destino Android net9.0-android y Windows net9.0-windows10.0.19041.0 cuando se compila en Windows.
RNF-02	Rendimiento	El HttpClient tiene un tiempo máximo de espera configurado de 60 segundos.
RNF-03	Seguridad	La comunicación con Pollinations se realiza mediante HTTPS y la clave se almacena usando SecureStorage.
RNF-04	Privacidad	La interfaz advierte que no se deben ingresar datos personales, contraseñas ni información confidencial.
RNF-05	Persistencia local	El historial debe permanecer en SQLite en el dispositivo y no se sincroniza con un servidor propio.
RNF-06	Mantenibilidad	Separación por Views, ViewModels, Services y Models mediante MVVM.
RNF-07	Usabilidad	La interfaz debe informar estados de carga y errores y disponer de textos accesibles en controles principales.
RNF-08	Confiabilidad	La aplicación debe gestionar errores HTTP, cancelación, ausencia de clave y falta de conexión sin cerrar inesperadamente la experiencia de usuario.

6.3 Restricciones y supuestos

- Se requiere conexión a Internet para generar respuestas; el historial local puede consultarse sin conexión.
- El usuario debe obtener y registrar una clave compatible con Pollinations. El código valida que comience por sk_ o pk_.
- El identificador del modelo es editable; que un modelo aparezca configurado no garantiza que el proveedor lo mantenga disponible.
- La longitud máxima de 120 caracteres está implementada tanto en la interfaz como en la lógica del servicio.
- El historial se guarda localmente en el archivo asistente.db3 ubicado en FileSystem.AppDataDirectory.

VII. ARQUITECTURA DE SOFTWARE

Se utiliza una arquitectura de presentación MVVM (Model-View-ViewModel) en una aplicación .NET MAUI. La interfaz XAML se enlaza con ViewModels; estos coordinan los Services; los servicios se encargan de la API externa, credenciales, conectividad y SQLite.

7.1 Vista lógica

```
Usuario
  |
  v
Views (XAML: Home, Chat, History, Detail, Settings)
  |  data binding / commands
  v
ViewModels (Home, Chat, History, Detail, Settings)
  |
  +--> AiService ------ HTTPS ------> Pollinations API
  +--> ApiKeyService ---------------> SecureStorage
  +--> DatabaseService ------------> SQLite local
  +--> ConnectivityService --------> Estado de red
  +--> Preferences ----------------> Modelo seleccionado

Models: ChatRequest, ChatResponse, ChatMessage, ConversationRecord, PromptTemplate
```

7.2 Componentes y responsabilidades

Componente	Responsabilidad observada
Views/	Pantallas XAML y code-behind. Presentan controles, enlazan propiedades y comandos.
ViewModels/	Estado de pantalla, validación de interacción, comandos asíncronos y coordinación entre servicios.
AiService.cs	Valida prompt y clave, construye el JSON, agrega Bearer, envía HTTP y deserializa la respuesta.
ApiKeyService.cs	Guarda, recupera y elimina la clave con SecureStorage.
DatabaseService.cs	Inicializa SQLite y realiza operaciones CRUD y búsqueda sobre ConversationRecord.
ConnectivityService.cs	Consulta Connectivity.Current.NetworkAccess para comprobar acceso a Internet.
Models/	Define contratos REST, registros locales y plantillas de prompt.
MauiProgram.cs	Registra HttpClient y dependencias mediante inyección de dependencias.

7.3 Flujo de una consulta

1. El usuario escribe una consulta o selecciona una plantilla en ChatPage.
2. ChatViewModel comprueba la longitud y la conectividad; activa el estado de carga.
3. AiService vuelve a validar el prompt, recupera la clave de SecureStorage y crea ChatRequest.
4. Se envía POST a chat/completions con Authorization: Bearer y Content-Type application/json.
5. Pollinations procesa la solicitud y devuelve un objeto JSON con choices[].message.content.
6. AiService extrae el primer contenido; ChatViewModel actualiza la interfaz e informa que la respuesta debe verificarse.
7. Al pulsar Guardar, ChatViewModel crea ConversationRecord y DatabaseService lo inserta en SQLite.

7.4 Persistencia y datos

Entidad / almacenamiento	Campos o contenido
ConversationRecord (SQLite)	Id autoincremental; Prompt; Answer; CreatedAt (UTC); Category; IsVerified.
ChatRequest (REST)	Model; Messages: lista de mensajes con Role y Content.
ChatResponse (REST)	Choices: lista de ChatChoice; cada elemento incluye Message.
SecureStorage	Clave identificada como pollinations_api_key.
Preferences	Identificador del modelo bajo la clave model; valor predeterminado observado: openai/gpt-5.4-nano.

VIII. DECISIONES TÉCNICAS

ID	Decisión	Justificación
DT-01	Uso de .NET MAUI y C#	Permite construir una aplicación nativa multiplataforma desde una base de código .NET. El proyecto configura Android y Windows.
DT-02	Patrón MVVM	Separa interfaz y lógica de presentación; CommunityToolkit.Mvvm 8.4.2 simplifica propiedades observables y comandos.
DT-03	Proveedor externo Pollinations	Evita implementar y operar un backend de inferencia propio. La aplicación utiliza un endpoint compatible con Chat Completions.
DT-04	HttpClient compartido	Se registra como singleton con BaseAddress https://gen.pollinations.ai/v1/ y timeout de 60 segundos.
DT-05	JSON tipado	ChatRequest y ChatResponse usan System.Text.Json y JsonPropertyName para mapear nombres del contrato.
DT-06	SQLite local	Permite conservar y consultar las conversaciones sin requerir una cuenta o servidor de base de datos.
DT-07	SecureStorage	Evita persistir la clave en texto plano dentro de Preferences o del historial.
DT-08	Preferences para modelo	Guarda la selección del modelo como una preferencia sencilla del dispositivo.

DT-09	Cancelación asíncrona	CancellationTokenSource permite interrumpir una consulta desde la interfaz.
DT-10	Validación de entrada	Se restringe el prompt a 5–120 caracteres y se validan prefijos/caracteres de la clave antes de guardarla.

IX. CONSUMO DEL SERVICIO REST

La aplicación consume la API de Pollinations con un contrato compatible con OpenAI Chat Completions. El servicio HTTP se configura en MauiProgram.cs. La operación funcional identificada es POST /chat/completions; al combinarse con la BaseAddress, la URL completa queda:

```
https://gen.pollinations.ai/v1/chat/completions
```

5.1 Solicitud

Elemento	Valor
Método	POST
URL	https://gen.pollinations.ai/v1/chat/completions
Protocolo	HTTPS
Autenticación	Authorization: Bearer <API_KEY>
Content-Type	application/json; charset=utf-8
Timeout del cliente	60 segundos
Validaciones previas	Prompt recortado de 5 a 120 caracteres; clave presente en SecureStorage.

Ejemplo de cuerpo enviado (los valores ilustran el contrato y no son credenciales reales):

```
{
  "model": "openai/gpt-5.4-nano",
  "messages": [
        {
        "role": "system",
        "content": "Eres un asistente académico. Responde de forma clara, indica incertidumbre y no inventes fuentes."
        },
        {
        "role": "user",
        "content": "Explica qué es la programación orientada a objetos."
```

```
    }
  ]
}
```

5.2 Respuesta exitosa esperada

El cliente deserializa un objeto con una colección choices y obtiene el contenido de message.content del primer elemento. El siguiente ejemplo ilustra los campos que consume el modelo C#; pueden existir campos adicionales en la respuesta real del proveedor.

```
{
  "choices": [
      {
      "message": {
      "role": "assistant",
      "content": "La programación orientada a objetos es un paradigma..."
      }
      }
  ]
}
```

Si choices no contiene contenido o el texto está vacío, AiService devuelve el mensaje local: “El servicio no devolvió contenido.”

5.3 Códigos HTTP y tratamiento

Código	Significado	Tratamiento/documentación	Estado en el código
200 OK	Solicitud procesada correctamente.	Deserializar JSON, leer choices[0].message.content y mostrar la respuesta.	Consumido por el código; contrato esperado.
400 Bad Request	Solicitud inválida, cuerpo incorrecto o parámetros no aceptados.	Se produce HttpRequestException; la interfaz informa el código HTTP. Revisar modelo y payload.	Posible respuesta estándar HTTP; no se maneja de forma específica.
401 Unauthorized	Clave ausente, inválida o no reconocida por el proveedor.	Mostrar aviso para revisar la clave en Configuración.	El ViewModel trata explícitamente Unauthorized.

403 Forbidden	Clave sin permiso, restricción o acceso denegado.	Mostrar aviso para revisar la clave/autorización.	El ViewModel trata explícitamente Forbidden.
404 Not Found	Ruta o recurso no disponible.	Mostrar código de error del servicio; revisar BaseAddress y ruta.	Posible respuesta HTTP.
429 Too Many Requests	Cuota, límite de frecuencia o capacidad excedida.	Mostrar código del servicio; esperar y reintentar manualmente más tarde.	No existe tratamiento específico ni Retry-After implementado.
5xx Server Error	Fallo temporal del proveedor.	Mostrar código; reintentar cuando el servicio se recupere.	No hay reintento automático.
Sin código / error de red	DNS, TLS, desconexión, timeout u otro fallo de transporte.	Captura HttpRequestException sin StatusCode o OperationCanceledException.	Tratamiento parcial implementado.

5.4 Encabezados y manejo de errores

```
Authorization: Bearer <clave configurada>
Content-Type: application/json
```

AiService ejecuta SendAsync(request, token) y EnsureSuccessStatusCode(). ChatViewModel y SettingsViewModel capturan HttpRequestException, OperationCanceledException, ArgumentException e InvalidOperationException. El mensaje mostrado al usuario se simplifica y no presenta el contenido de la clave.

5.5 Consideraciones de contrato

- El identificador del modelo es dinámico y procede de Preferences. El valor predeterminado del repositorio es openai/gpt-5.4-nano.
- El contenido de la consulta se envía al proveedor externo. No debe asumirse que permanece exclusivamente en el dispositivo.
- La aplicación toma el primer elemento de choices; no procesa varias alternativas ni métricas de uso.
- El cliente no implementa streaming; espera la respuesta HTTP completa.

· Los códigos concretos, mensajes, límites y disponibilidad de modelos dependen del proveedor y pueden cambiar.

VIII. MATRIZ DE RIESGOS DE PRIVACIDAD, SEGURIDAD Y USO RESPONSABLE

Escala cualitativa: Probabilidad (P) e impacto (I) bajo, medio o alto. Nivel resultante orientativo: bajo, medio, alto o crítico. La matriz incluye controles existentes y acciones recomendadas; no equivale a una auditoría de seguridad formal.

ID	Tipo	Riesgo	P	I	Nivel	Control/mitigación
R-01	Privacidad	El usuario introduce datos personales, contraseñas o información confidencial en el prompt.	Media	Alto	Alto	Avisos visibles en Inicio, Consultar y Configuración. Reforzar con ejemplos, minimización y advertencia previa al envío.
R-02	Privacidad	La consulta y respuesta se transmiten a un proveedor externo.	Alta	Alto	Alto	Informar qué datos se envían, finalidad y proveedor; revisar política de retención y términos de Pollinations; no enviar datos sensibles.
R-03	Seguridad	Exposición de la clave API por captura, dispositivo comprometido, backup o registro accidental.	Media	Alto	Alto	SecureStorage; campo enmascarado; limpiar campo después de guardar; nunca registrar ni versionar la clave. Evaluar restricciones de backup y rotación.
R-04	Seguridad	Uso indebido de una clave filtrada o publicación	Media	Alto	Alto	No incluir secretos en código ni commits; revocar y regenerar ante filtración; revisar historial del repositorio y permisos de la clave.

		accidental en el repositorio.				
R-05	Seguridad	Ataque de intermediario o manipulación del tráfico.	Baja	Alto	Medio	Mantener HTTPS y validación TLS del sistema; no aceptar certificados inválidos; evitar endpoints HTTP.
R-06	Seguridad	Modelo o endpoint no disponible, límites excedidos o abuso que genere costos/consumo de cuota.	Media	Medio	Medio	Mostrar errores 429/5xx; considerar cuotas, límites de uso, control de frecuencia y reintentos con backoff.
R-07	Privacidad	Historial local expuesto a otra persona que accede al dispositivo o a copias de seguridad.	Media	Alto	Alto	Ofrecer borrado; proteger el dispositivo con bloqueo/cifrado del sistema; informar que el historial queda en el dispositivo. Evaluar cifrado de SQLite.
R-08	Uso responsable	El usuario toma la respuesta de IA como verdad o referencia académica verificada.	Alta	Alto	Alto	Aviso de verificación; indicador IsVerified manual; instrucción de contrastar con fuentes confiables. Aclarar que la marca no valida automáticamente.
R-09	Uso responsable	La IA genera información falsa, sesgada, desactualizada o referencias inventadas.	Alta	Alto	Alto	Instrucción de sistema para no inventar fuentes; mensajes visibles de cautela; contrastar información importante.

R-10	Uso responsable	Uso de la herramienta para contenido dañino, plagio o sustitución del aprendizaje.	Media	Medio	Medio	Definir uso como apoyo educativo, no como sustituto del criterio; citar y revisar fuentes; establecer normas académicas.
R-11	Disponibilidad	Pérdida o corrupción de historial local o fallo de almacenamiento.	Baja	Medio	Bajo	Mantenimiento de SQLite, pruebas CRUD y mecanismo futuro de exportación/backup bajo control del usuario.
R-12	Seguridad	La validación de prefijo sk_/pk_ se interpreta como prueba de validez de una clave.	Media	Medio	Medio	Aclarar que solo es una comprobación superficial; la validez real se confirma con la API. No registrar el secreto.

8.1 Reglas de uso responsable para el usuario

· No introducir nombres completos, direcciones, números de identificación, datos financieros, historias clínicas, contraseñas ni información privada de terceros.

· Usar la IA como apoyo para estudiar, no como fuente definitiva ni sustituto del docente, bibliografía o criterio profesional.

· Contrastar hechos, cifras, citas y referencias con fuentes confiables y actualizadas.

· Marcar “Respuesta verificada” únicamente después de revisar la información por cuenta propia. El indicador es una declaración del usuario, no una verificación automática.

· No compartir la clave API y eliminarla de Configuración cuando se deje de utilizar el dispositivo o la aplicación.

IX. MANUAL DE USUARIO

9.1 Requisitos previos

- Dispositivo Android compatible con el mínimo configurado por el proyecto (API 21 o superior), **O** Windows compatible con el destino configurado.
- Conexión a Internet para realizar consultas.
- Clave API de Pollinations válida y compatible con el endpoint utilizado.
- Aplicación instalada y abierta.

9.2 Primer uso: configurar la clave

8. Abrir la pestaña Configuración.
9. En “Clave de la API”, pegar la clave entregada por el proveedor. El campo oculta los caracteres.
10. Pulsar “Guardar clave”. La aplicación valida que no esté vacía, que use caracteres ASCII visibles y que comience por sk_ o pk_.
11. Comprobar que el estado cambie a “Clave registrada: Sí”. La clave no se vuelve a mostrar.
12. En “Modelo de IA”, conservar el identificador predeterminado o escribir uno válido y pulsar “Guardar modelo”.
13. Pulsar “Probar conexión”. La aplicación realiza una consulta breve y muestra el resultado.

Privacidad No pegues claves en chats, capturas, documentos compartidos o repositorios públicos. Si sospechas que una clave se expuso, revócala en el proveedor y registra una nueva.

9.3 Realizar una consulta

14. Abrir la pestaña Consultar.
15. Opcionalmente, elegir una plantilla: explicar, resumir, crear ejemplos, preguntas de práctica o comparar conceptos. El texto de la plantilla se carga en el editor y puede modificarse.
16. Escribir una pregunta clara y específica. Debe tener entre 5 y 120 caracteres.

17. Pulsar “Consultar”. Mientras se procesa, se muestra el indicador de actividad y el estado “Consultando el servicio...”.

18. Leer la respuesta en el área inferior. La aplicación muestra un recordatorio para verificar la información.

19. Para interrumpir una solicitud en curso, pulsar “Cancelar”.

20. Para conservar la conversación, pulsar “Guardar”. La consulta, respuesta, fecha y categoría se almacenan en el dispositivo.

9.4 Consultar y administrar el historial

21. Abrir la pestaña Historial. Se muestran las conversaciones guardadas, primero las más recientes.

22. Escribir palabras en “Buscar en el historial” para filtrar por texto de consulta o respuesta.

23. Seleccionar una tarjeta para abrir el detalle.

24. En el detalle se muestran consulta, respuesta, fecha y categoría.

25. Activar “Respuesta verificada” solo después de contrastar los datos con fuentes confiables y pulsar “Guardar cambios”.

26. Para eliminar un registro, pulsar “Eliminar” y confirmar. La eliminación también está disponible desde la pantalla de detalle.

9.5 Eliminar la clave o cambiar el modelo

Desde Configuración se puede cambiar el identificador del modelo y guardar la nueva preferencia. Para retirar la credencial, pulsar “Eliminar clave” y confirmar en el diálogo. La aplicación actualiza el estado a “Clave registrada: No”.

9.6 Mensajes frecuentes

Mensaje o situación	Qué hacer
Configure la clave de la API.	Abrir Configuración y registrar una clave.
La consulta es demasiado corta.	Escribir al menos 5 caracteres útiles.

La consulta no puede superar 120 caracteres.	Reducir el texto; el límite está definido por el proyecto.
Sin conexión a internet.	Comprobar Wi-Fi o datos móviles y volver a intentar.
Error de autorización.	Verificar la clave, permisos y vigencia con el proveedor.
Tiempo agotado / consulta cancelada.	Revisar la conexión y volver a enviar la solicitud.
Error del servicio: [código].	Consultar el significado del código HTTP; puede tratarse de un problema del proveedor, cuota o configuración.
No hay consultas guardadas.	Enviar una consulta y pulsar Guardar para crear el primer registro.

X. MANUAL TÉCNICO BREVE

10.1 Stack tecnológico

Tecnología / dependencia	Versión o configuración	Uso
.NET	9	Framework de la aplicación.
.NET MAUI	MauiVersion administrada por SDK	UI móvil y multiplataforma con XAML.
C#	Nullable e implicit usings habilitados	Lógica de aplicación.
CommunityToolkit.Mvvm	8.4.2	ObservableObject, ObservableProperty, RelayCommand.
sqlite-net-pcl	1.11.285	Persistencia SQLite.
Microsoft.Extensions.Logging.Debug	9.0.8	Logging de depuración.
HttpClient / System.Text.Json	Incluidos en .NET	Consumo REST y serialización JSON.

10.2 Estructura relevante del proyecto

```
AsistenteIA/
|-- Models/
|   |-- ChatModels.cs
|   |-- ConversationRecord.cs
```

```
|   `-- PromptTemplate.cs
|-- Services/
|   |-- AiService.cs
|   |-- ApiKeyService.cs
|   |-- ConnectivityService.cs
|   `-- DatabaseService.cs
|-- ViewModels/
|   |-- ChatViewModel.cs
|   |-- DetailViewModel.cs
|   |-- HistoryViewModel.cs
|   |-- HomeViewModel.cs
|   `-- SettingsViewModel.cs
|-- Views/
|   |-- ChatPage.xaml
|   |-- DetailPage.xaml
|   |-- HistoryPage.xaml
|   |-- HomePage.xaml
|   `-- SettingsPage.xaml
|-- Platforms/
|-- Resources/
|-- App.xaml
|-- AppShell.xaml
|-- MauiProgram.cs
`-- AsistenteIA.csproj
```

10.3 Entorno de desarrollo

El entorno recomendado es SDK de .NET 9, Visual Studio 2022 con la carga de trabajo .NET MAUI y Android SDK o entorno Windows compatible.

10.4 Instalación y compilación

```
git clone https://github.com/Cristianm23/AsistenteIA.git
 cd AsistenteIA
 dotnet restore AsistenteIA.sln
 dotnet build AsistenteIA/AsistenteIA.csproj -f net9.0-android
```

Para Windows, compilar desde un equipo Windows que tenga instaladas las herramientas MAUI y el targeting pack requerido:

```
dotnet build AsistenteIA/AsistenteIA.csproj -f net9.0-windows10.0.19041.0
```

La ejecución puede realizarse desde Visual Studio seleccionando un emulador Android, dispositivo conectado o destino Windows disponible.

10.5 Configuración de servicios

En MauiProgram.cs se registra un HttpClient singleton con BaseAddress https://gen.pollinations.ai/v1/ y Timeout de 60 segundos. AiService, ApiKeyService, DatabaseService y ConnectivityService también se registran como singleton; ViewModels y páginas se registran como transient.

Para modificar el proveedor o la ruta REST, revisar AiService.cs y la BaseAddress de MauiProgram.cs. Para cambiar el esquema local, revisar ConversationRecord.cs y DatabaseService.cs. Si se agregan o modifican columnas, implementar una estrategia de migración compatible con bases ya existentes.

10.6 Almacenamiento local

```
Ruta SQLite: Path.Combine(FileSystem.AppDataDirectory, "asistente.db3")
Tabla: ConversationRecord
Clave segura: pollinations_api_key
Preferencia de modelo: model
```

DatabaseService inicializa la conexión de forma diferida, crea la tabla y ofrece Insert, Select, búsqueda con Contains, consulta por Id, Update y Delete.

10.7 Puntos de mantenimiento

- No incluir claves, tokens o credenciales en código fuente, archivos de configuración versionados, logs o capturas de pantalla.
- Mantener sincronizados los límites de longitud de prompt entre ChatPage.xaml, ChatViewModel y AiService.
- Usar CancellationToken en operaciones de red y conservar el timeout controlado.
- Agregar pruebas unitarias para validación, deserialización, errores HTTP y persistencia.
- Si se actualiza el contrato del proveedor, verificar ChatRequest, ChatResponse y la extracción de choices[0].message.content.
- Evitar registrar prompts o respuestas completos en logs; pueden contener información sensible.

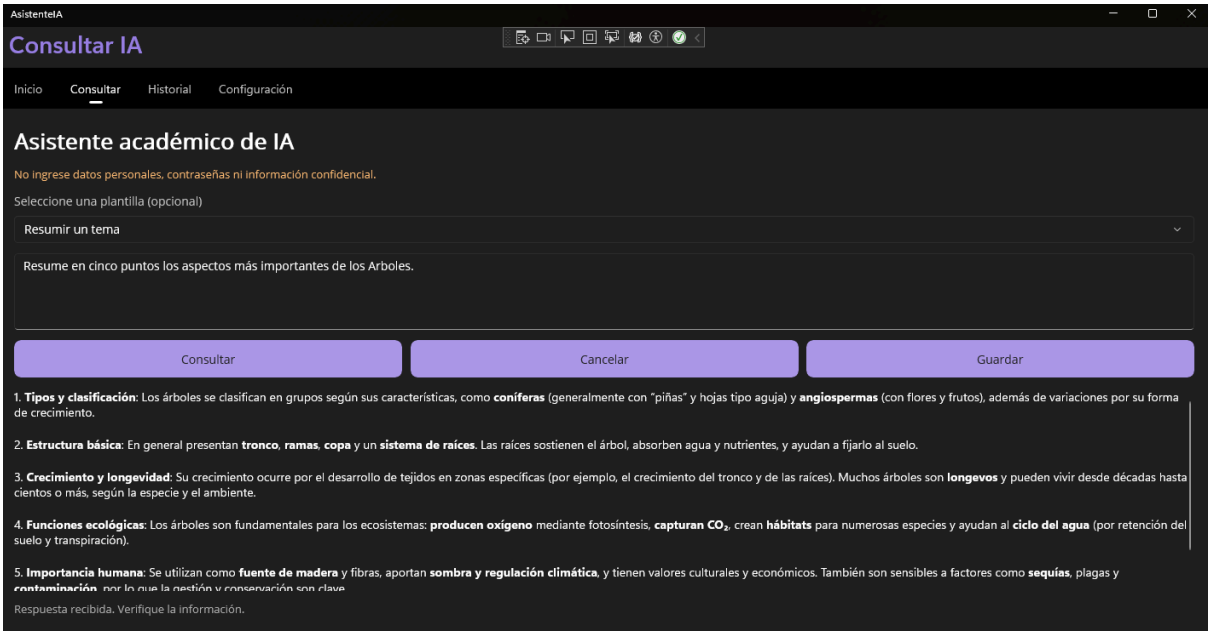
XI. PRUEBAS Y CRITERIOS DE ACEPTACIÓN

ID	Escenario	Criterio de aceptación

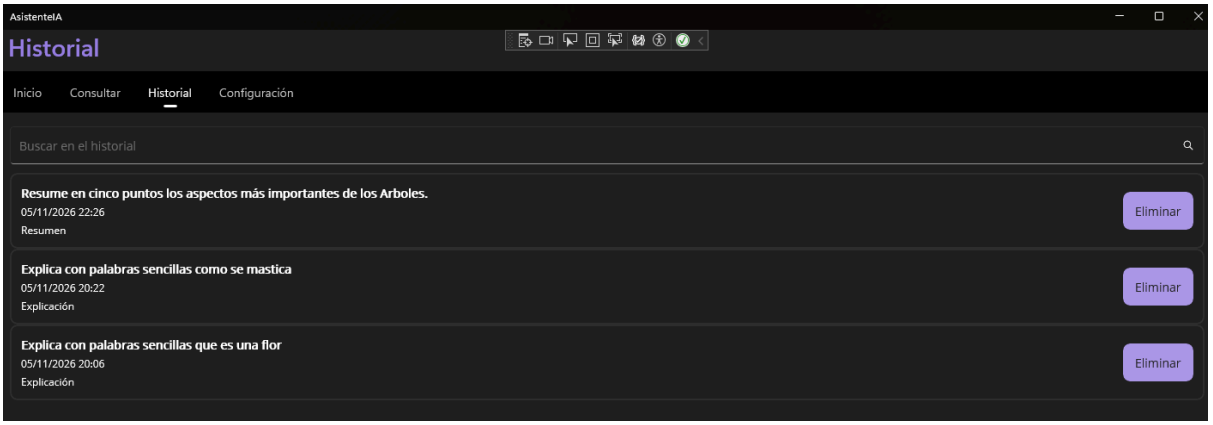
CP-01	Consulta de 4 caracteres	No se envía solicitud; mensaje de longitud mínima.
CP-02	Consulta entre 5 y 120 caracteres	Se permite iniciar la consulta si hay clave y conectividad.
CP-03	Consulta de más de 120 caracteres	Se bloquea en UI/lógica y no se envía al proveedor.
CP-04	Sin clave guardada	Se informa que debe configurar la clave; no se genera solicitud autenticada.
CP-05	Clave inválida o sin permiso	Se presenta el mensaje de autorización y no se muestra el secreto.
CP-06	Sin Internet	Se muestra aviso y no se inicia el envío.
CP-07	Respuesta válida con choices	Se muestra el contenido del primer choice.
CP-08	Respuesta con choices vacío	Se muestra el texto de ausencia de contenido.
CP-09	Cancelar consulta	La operación se cancela o termina por timeout y se restablece IsBusy.
CP-10	Guardar consulta	El registro aparece en Historial después de guardarse.
CP-11	Buscar historial	El filtro encuentra coincidencias en prompt o respuesta.
CP-12	Eliminar historial	Tras confirmar, el registro desaparece de SQLite y de la lista.
CP-13	Marcar como verificada	El valor IsVerified persiste después de guardar cambios y volver a abrir el detalle.
CP-14	Guardar/eliminar clave	El estado cambia sin mostrar el contenido de la clave.
CP-15	Prueba de conexión	Se informa éxito solo si la llamada al endpoint completa correctamente.

XII. 10 Pruebas de la app

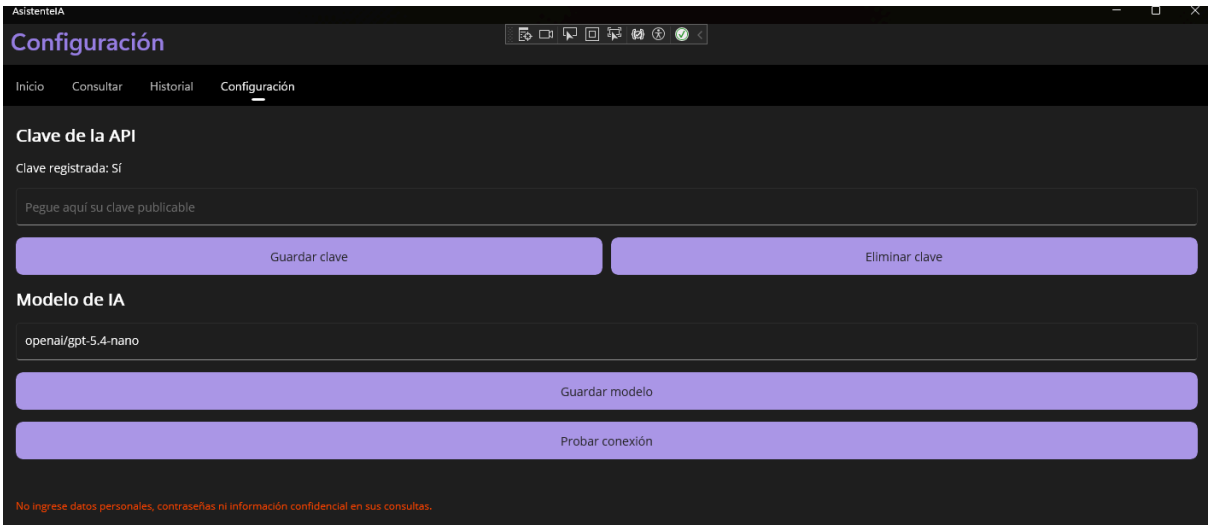
Prueba de consulta



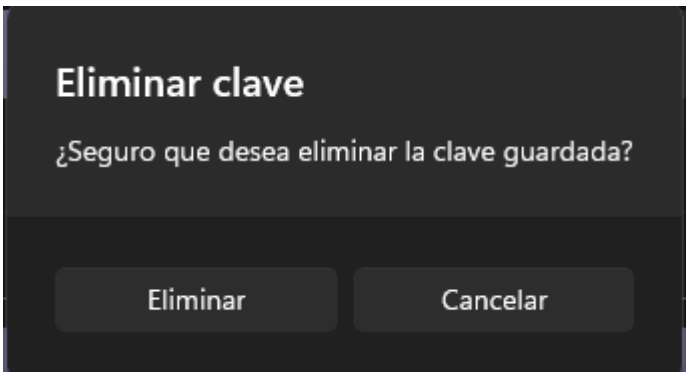
Prueba historial con guardado



Prueba de configuracion y clave api



Autenticacion de borrado de api



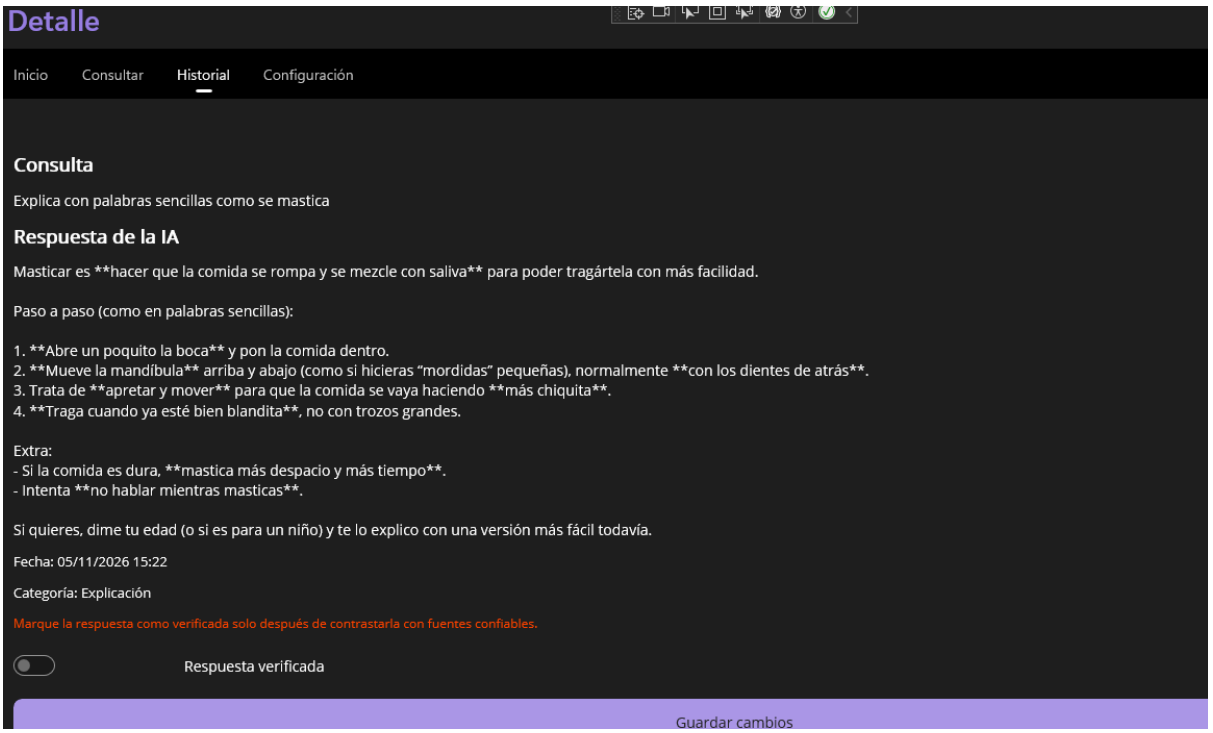
Prueba de conexion



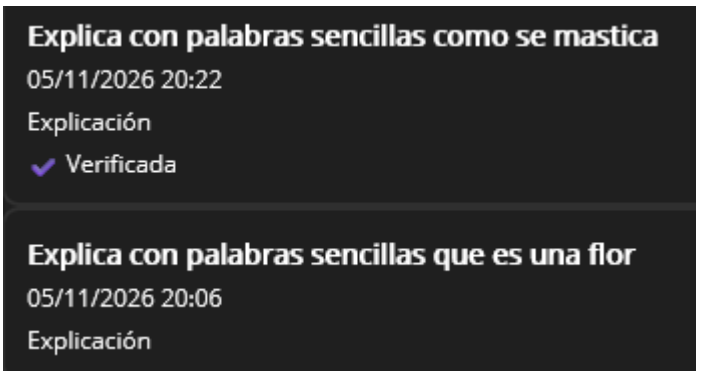
Prueba pantalla responsive:



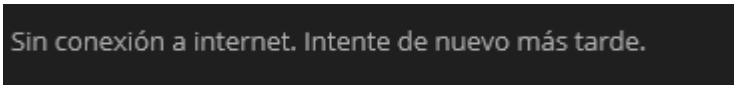
Prueba detalle de el historial con opcion a verificar la informacion dada



Diferencia entre la verificacion o no

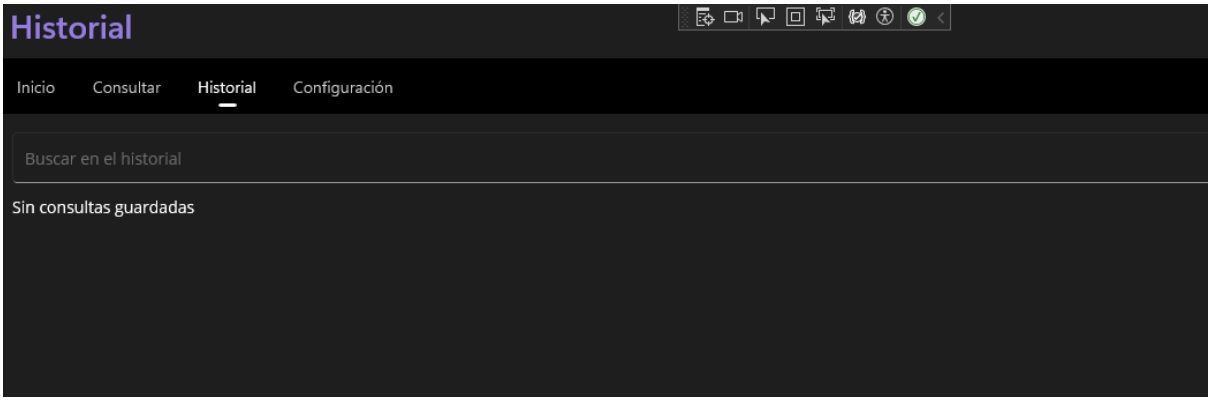


Prueba Excepciones





Prueba eliminación historial



XIII. REFERENCIAS

- Repositorio: https://github.com/Cristianm23/AsistenteIA
- README.md - descripción, funcionalidades, arquitectura declarada e instrucciones de instalación.
- AsistenteIA/AsistenteIA.csproj - frameworks objetivo y dependencias NuGet.
- AsistenteIA/MauiProgram.cs - configuración de HttpClient e inyección de dependencias.
- AsistenteIA/Services/AiService.cs - validación, construcción y envío de solicitud REST.
- AsistenteIA/Services/ApiKeyService.cs - almacenamiento seguro de la clave.
- AsistenteIA/Services/DatabaseService.cs - persistencia SQLite.
- AsistenteIA/Services/ConnectivityService.cs - verificación de conectividad.
- AsistenteIA/Models/ChatModels.cs y ConversationRecord.cs - modelos de datos.
- AsistenteIA/ViewModels/ y AsistenteIA/Views/ - comportamiento y pantallas de la aplicación.
- AsistenteIA/Platforms/Android/AndroidManifest.xml - permisos de red y acceso a estado de conexión.

Documentación externa de referencia técnica: Microsoft Learn para .NET MAUI, HttpClient, SecureStorage, Preferences y SQLite; documentación oficial de Pollinations para el endpoint y autenticación vigentes. Los contratos y políticas del proveedor deben verificarse nuevamente antes de desplegar una versión pública.

Fin del documento